

# Análisis y Diseño con Patrones

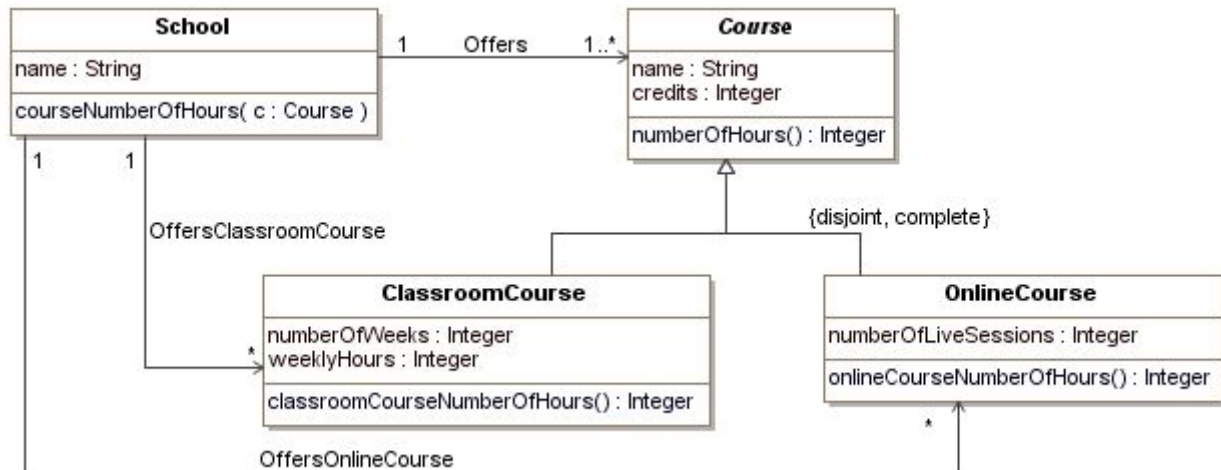
## PEC 1: Principios de Diseño y Patrones de Análisis

### Pregunta 1 (30%)

Queremos desarrollar un software que implementa un sistema para gestionar la información de los cursos que ofrecen varios centros docentes. El sistema registra centros docentes que tienen un nombre que los identifica. Los centros docentes ofrecen cursos. Los cursos tienen un nombre, unos créditos asignados y pueden ser presenciales o virtuales. Los cursos presenciales registran el número de horas semanales del curso y el número de semanas del curso. Los cursos virtuales registran el número de sesiones en directo que se realizan durante todo el curso. Cada sesión en directo es de una hora.

Nos han propuesto diseñar una operación para calcular el número de horas totales que los estudiantes tienen que dedicar a cada curso. A continuación, disponéis de una parte del diagrama de clases del sistema, así como del diagrama de secuencia de la operación *courseNumberOfHours(c: Course):Integer* de la clase *School*. Esta operación tiene como precondición que el curso *c* es uno de los cursos ofrecidos por el centro sobre el que se invoca la operación. La operación calcula el número de horas que un estudiante tiene que dedicar al curso *c* en función del tipo de curso. Si el curso es presencial, el número de horas serán las horas semanales multiplicadas por el número semanas. Si el curso es virtual, el número de horas será igual al número de sesiones en directo.

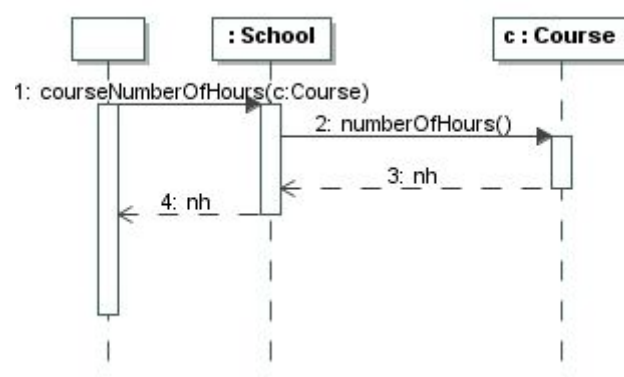
#### Diagrama de clases

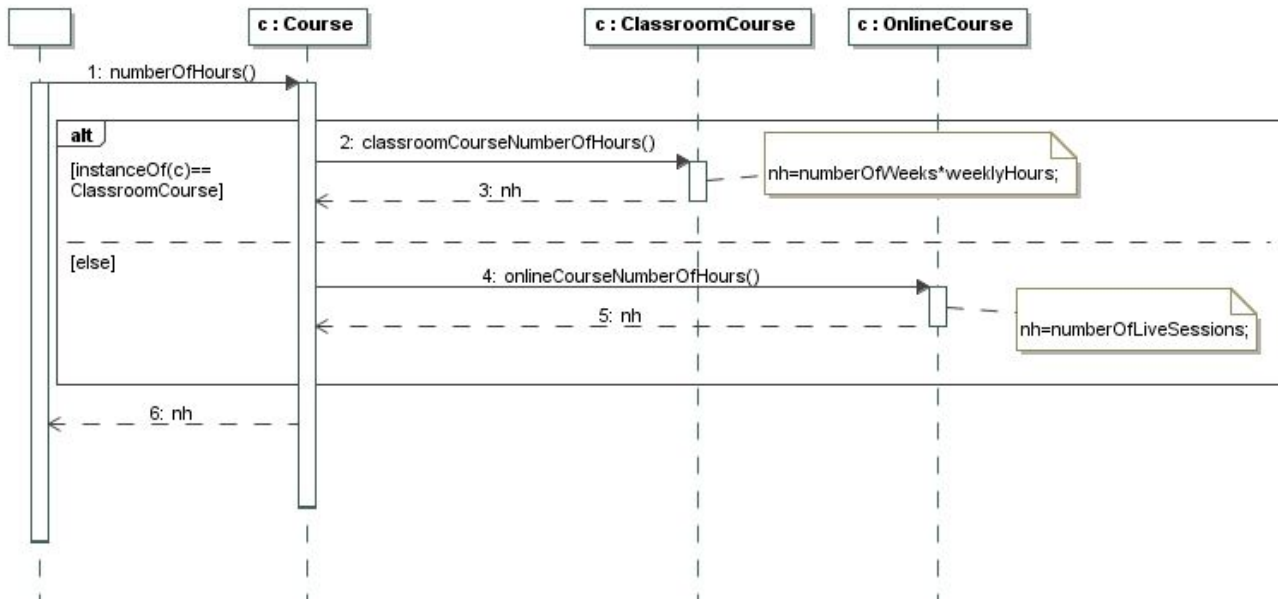


### Restricciones de integridad:

- Un centro se identifica por su nombre.
- Un curso se identifica por su nombre y el nombre del centro.
- Los cursos presenciales de un centro tienen que ser un subconjunto de los cursos que ofrece un centro.
- Los cursos virtuales de un centro tienen que ser un subconjunto de los cursos que ofrece un centro.

### Diagrama de secuencia de la operación `School::courseNumberOfHours(c: Course):Integer`





Se pide:

- En este ejemplo se violan de dos principios de diseño. ¿Cuáles son y por qué?
- ¿Qué consecuencias tienen estas violaciones?

## Solución

- Los dos principios de diseño que se violan en este ejemplo son:
  - Bajo Acoplamiento. Si nos fijamos en el diagrama de clases, la clase *School* está acoplada con la clase *Course* y con las clases *ClassroomCourse* y *OnlineCourse* puesto que existe una asociación navegable entre la clase *School* y el resto de clases. Esto quiere decir que la clase *School* tiene conocimiento y, por lo tanto, acoplamiento con las 3 clases. Se podría mantener la misma información con un acoplamiento más bajo (ver solución de la pregunta 2.a).
  - Abierto-cerrado. Si nos fijamos en el diagrama de secuencia de la operación *Course::numberOfHours():Integer* que es invocada desde la operación *School::courseNumberOfHours(c: Course): Integer* podemos observar que esta operación no está cerrada a la modificación. Por ejemplo, si añadimos otro tipo de curso y queremos calcular el número de horas del curso, habría que modificar la operación *Course::numberOfHours():Integer* añadiendo un nuevo caso para tratar el nuevo tipo de curso. Esto quiere decir que una operación que teníamos implementada hay que modificarla (no está cerrada a la modificación). Satisfacer el principio querría decir que añadir un nuevo tipo de curso tendría que suponer una extensión de nuestro sistema, pero no una modificación.

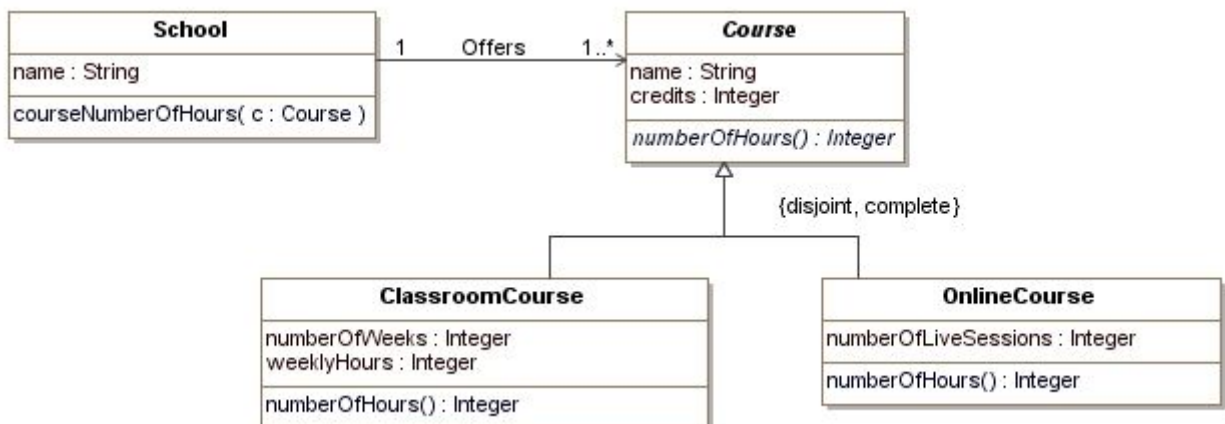
- b) Las violaciones de los dos principios anteriores suponen una dificultad en las modificaciones de nuestro sistema.
- Mantener un acoplamiento alto implica que cambios que se pueden producir en las clases *Course*, *ClassRoomCourse* y *OnlineCourse* pueden implicar cambios en la clase *School*.
  - Como hemos descrito en el apartado anterior una violación del principio Abierto-cerrado puede implicar que un cambio en nuestro sistema (por ejemplo, añadir un nuevo tipo de curso), sea más difícil de añadir. Es más fácil extender un sistema (por ejemplo, añadir una nueva clase con sus operaciones) que modificar las operaciones de una clase existente en nuestro sistema.

## Pregunta 2 (30%)

Propón como modificarías el diagrama de clases y la implementación de la operación `courseNumberOfHours(c: Course): Integer` para satisfacer los dos principios de diseño. Razona, también, como la solución supera las consecuencias negativas del apartado *b* de la pregunta anterior.

### Solución

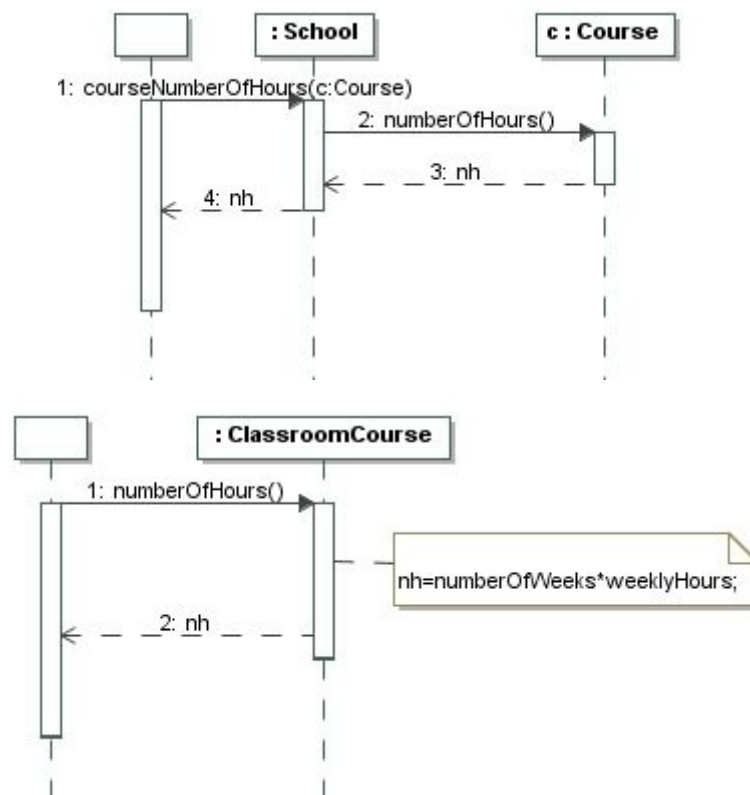
- a) Para reducir el acoplamiento modificaremos el diagrama de clases de diseño eliminando las asociaciones navegables entre la clase *School* y las clases *ClassroomCourse* y *OnlineCourse* y las dos últimas restricciones de integridad. La eliminación de estas asociaciones no cambia la representación de la información del sistema que estamos desarrollando. A continuación, disponéis del nuevo diagrama de clases:

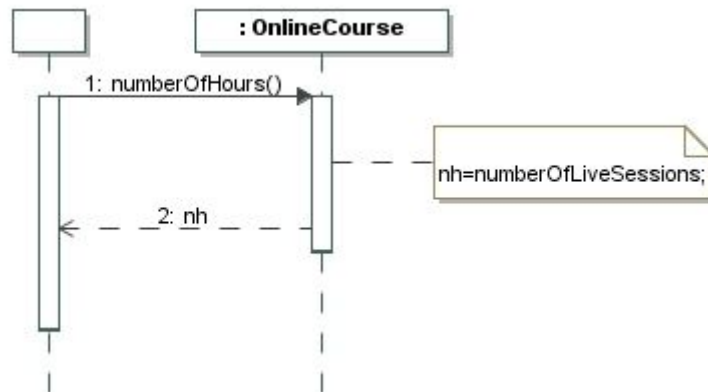


### Restricciones de integridad:

- Un centro se identifica por su nombre.

- Un curso se identifica por su nombre y el nombre del centro.
- b) Para satisfacer el principio de Abierto-cerrado tenemos que modificar la operación *Course::numberOfHours():Integer* que es invocada desde la operación *School::courseNumberOfHours(c: Course): Integer*. La modificación consistirá en: 1) definir la operación como abstracta y 2) sustituir la alternativa de la operación por la utilización del polimorfismo. El uso del polimorfismo favorece la satisfacción de este principio de diseño. A continuación, disponéis del nuevo diagrama de secuencia de la operación:





Fijaros que con el nuevo diseño de la operación, si añadimos otro tipo de curso y queremos calcular el número de horas del curso, no habría que modificar la operación

*Course::numberOfHours():Integer* puesto que es abstracta. Solo habría que añadir la nueva clase y añadir una nueva operación que implemente la operación *NumberOfHours():Integer*. Por lo tanto, la operación está cerrada a la modificación y abierta a la extensión.

## Pregunta 3 (20%)

Queremos desarrollar un software para la LFP (Liga de Fútbol Profesional) que implementa un sistema para grabar los jugadores que están o han sido contratados por los equipos de fútbol. Cada equipo ficha a sus jugadores en una fecha de inicio y se mantienen al equipo hasta que se acaba su contrato, son despedidos o son cedidos. En un momento determinado, un jugador solo puede jugar en un equipo. Eso sí, un jugador puede estar en un mismo equipo en varios periodos, por ejemplo, puede ser fichado por un equipo, cedido a otro equipo y después volver al equipo inicial.

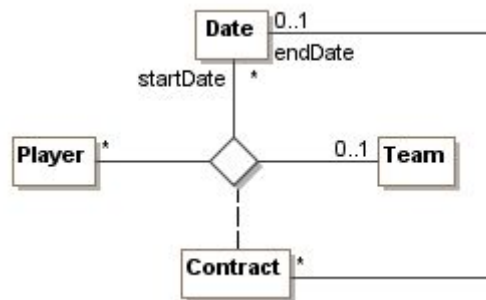
Nos proponemos desarrollar el módulo que implementa este software. Antes de hacer el diagrama estático de análisis, se deben revisar los patrones de análisis por si encontramos alguno que pueda ayudar a resolver el problema.

Indica qué patrón de análisis habría que tener especialmente en cuenta y justifica brevemente por qué. Propón un diagrama estático de análisis donde se aplique el patrón de análisis identificado por los requisitos descritos.

Nota: Si hace falta, puedes hacer alguna variación sobre el patrón tal como está explicado en los materiales para poder satisfacer todos los requisitos enumerados.

## Solución

En este caso podemos aplicar el patrón "Asociación histórica", puesto que nos permite mantener un histórico de los jugadores que están contratados por los equipos en diferentes rangos de fechas.



#### Restricciones de integridad:

- Un jugador no puede tener contratos en periodos solapados.

Hemos asignado una fecha de fin en la clase asociativa para poder grabar los periodos en los que los jugadores están contratados en sus equipos, de forma que podamos controlar y evitar que un jugador pueda estar contratado en diferentes equipos durante un mismo periodo.

## Pregunta 4 (20%)

Queremos desarrollar un sistema software para transformar documentos JSONs a otros formatos. En concreto el sistema tiene que grabar que los documentos JSON definen un objeto JSON. Este objeto tiene atributos que pueden ser de tres tipos: primitivos, arrays y referencias a objetos JSON. Los arrays pueden ser de tipos primitivos, de referencias a objetos o bien de arrays.

Indica qué patrón de análisis habría que tener especialmente en cuenta y justifica brevemente por qué. Propón un diagrama estático de análisis donde se aplique el patrón de análisis identificado en los requisitos descritos.

Nota: Si hace falta, puedes hacer alguna variación sobre el patrón tal como está explicado en los materiales para poder satisfacer todos los requisitos enumerados.

## Solución

En este caso podemos aplicar el patrón "Objeto Compuesto" para representar que los arrays pueden ser de tipo primitivos, referencias a objetos o arrays. Hemos modificado ligeramente el

patrón usando una asociación entre *Array* y *Type* en vez de una composición puesto que el *Type* puede existir con independencia de la existencia del array.

